

AI/ML Internship – Day 2 Task

Loya Pranay (pranay8679@gmail.com)

Role: AI/ML Intern

Date: 12-05-2026

Project Name: Pose Detection using MediaPipe

1. Objective

The objective of Day 2 was to understand the basics of computer vision, body tracking, and human pose detection using MediaPipe and OpenCV. The task also included setting up the AI environment, studying human pose landmarks, and running a real-time pose detection system using a webcam.

2. AI Environment Setup

Installed Tools

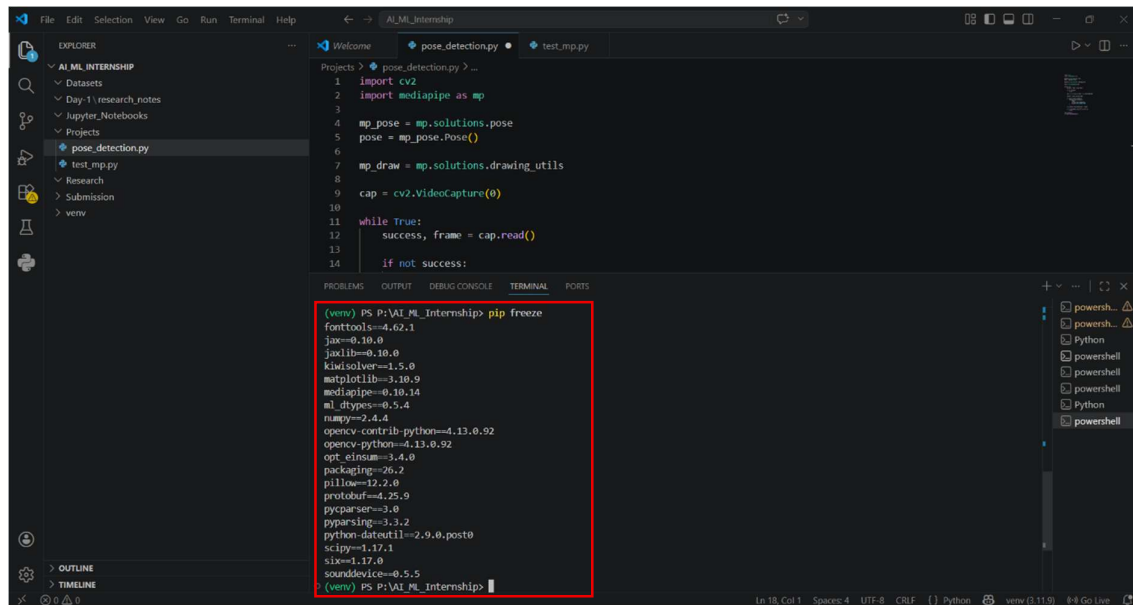
Tool	Purpose
Python	Programming Language
TensorFlow	AI/Deep Learning Framework
OpenCV	Computer Vision Processing
MediaPipe	Pose Detection
VS Code	Development Environment

Installation Commands Used

```
pip install tensorflow
```

```
pip install opencv-python
```

```
pip install mediapipe
```



3. Virtual Environment Setup

A separate Python virtual environment (venv) was created to avoid package conflicts and maintain a clean AI development workspace.

Commands Used

```
python -m venv venv
```

```
.\venv\Scripts\activate
```

Why Virtual Environments Are Useful

A virtual environment is an isolated Python workspace used to manage project-specific libraries and dependencies separately from the main system Python installation.

In AI and Machine Learning projects, multiple libraries such as TensorFlow, OpenCV, MediaPipe, and NumPy are required. Different projects may require different versions of these libraries. Installing everything directly into the system Python can create version conflicts and dependency issues.

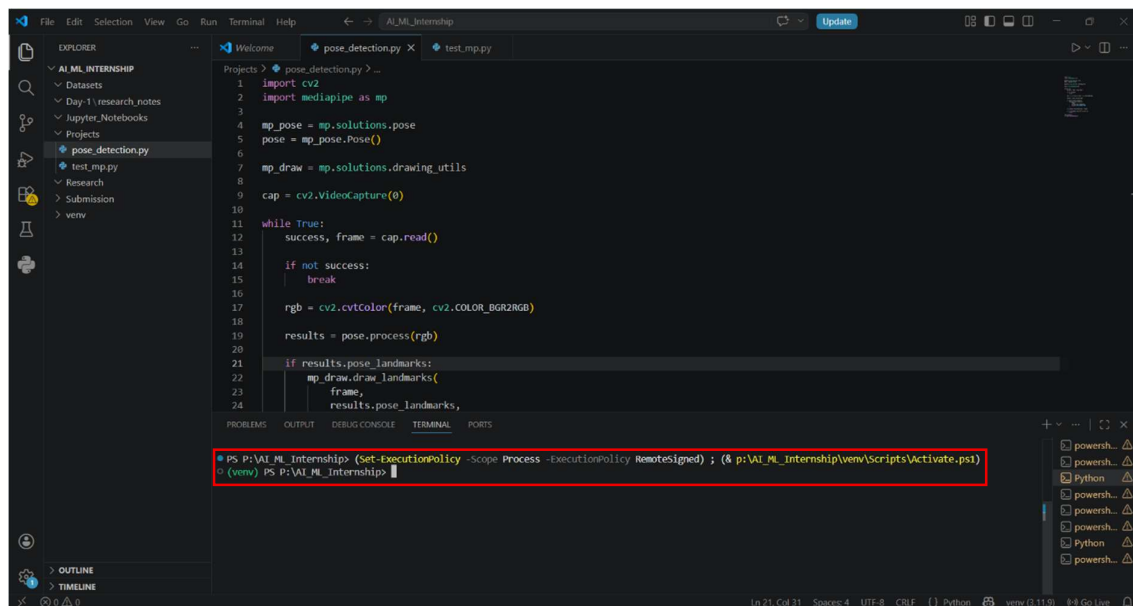
How Package Isolation Helps AI Projects

Package isolation means that all required libraries for a project are installed inside a separate environment instead of the global system Python installation.

In AI/ML development, libraries such as:

- TensorFlow
- OpenCV
- MediaPipe
- NumPy

often depend on specific compatible versions. If incompatible versions are installed globally, the project may fail with import errors or dependency conflicts.



4. Understanding Human Pose Detection

What is Pose Detection?

Pose detection is a computer vision technique that allows AI systems to identify and track human body positions using images or videos.

The AI system detects body key points called landmarks and connects them to form a skeletal structure.

Human Pose Landmarks

MediaPipe detects multiple body landmarks such as:

- nose
- eyes
- shoulders
- elbows
- wrists
- hips
- knees
- ankles

These landmarks help AI understand:

- posture
- movement
- gestures
- body orientation

Applications of Pose Detection

- Fitness tracking
- Yoga posture correction
- Sports analysis
- Gesture recognition
- Virtual try-on systems
- Healthcare monitoring
- Motion tracking

5. Body Measurement Basics

Body measurement AI uses pose landmarks and distance estimation techniques to approximate body proportions digitally.

Measurements Estimated

- Shoulder width
- Height
- Waist size
- Hip measurements
- Arm length

Uses

- Fashion size recommendation
- Tailoring
- Virtual fitting rooms

6. MediaPipe Pose Detection Workflow

Workflow Steps

Step 1 — Capture Webcam Input

OpenCV captures video frames using webcam.

Step 2 — Convert BGR to RGB

MediaPipe processes RGB images.

Step 3 — Detect Human Pose

MediaPipe identifies body landmarks.

Step 4 — Draw Landmarks

Detected landmarks are connected visually.

Step 5 — Display Output

Real-time skeletal tracking is displayed.

7. Code Implementation

MediaPipe Real-Time Pose Detection Code

```
import cv2
import mediapipe as mp

mp_pose = mp.solutions.pose
pose = mp_pose.Pose()

mp_draw = mp.solutions.drawing_utils

cap = cv2.VideoCapture(0)

while True:
    success, frame = cap.read()

    if not success:
        break

    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    results = pose.process(rgb)

    if results.pose_landmarks:
        mp_draw.draw_landmarks(
            frame,
            results.pose_landmarks,
```

```

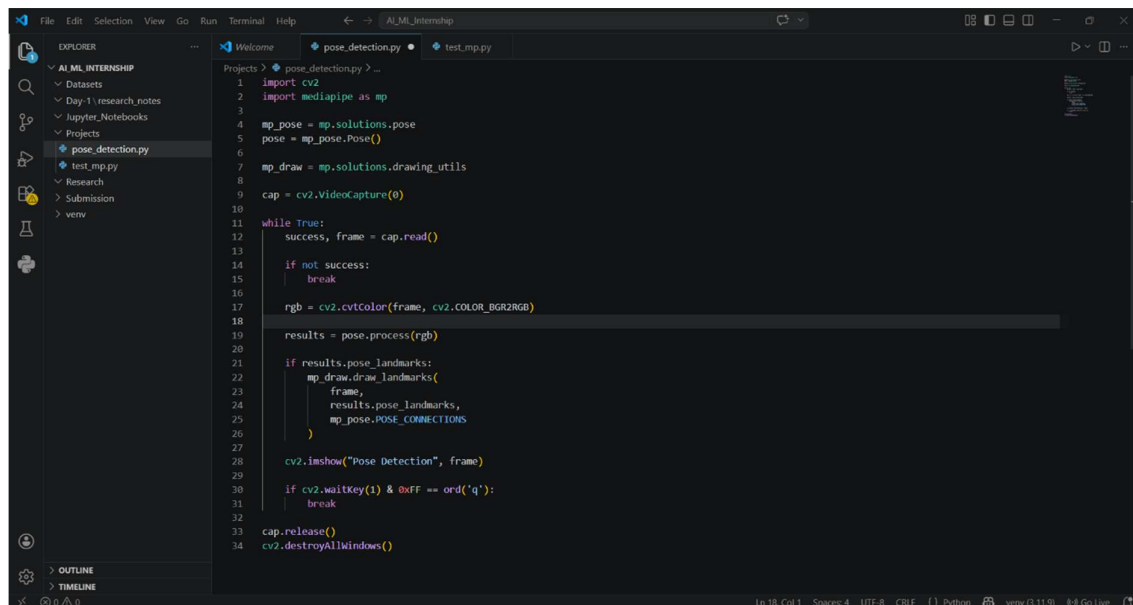
        mp_pose.POSE_CONNECTIONS
    )

    cv2.imshow("Pose Detection", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

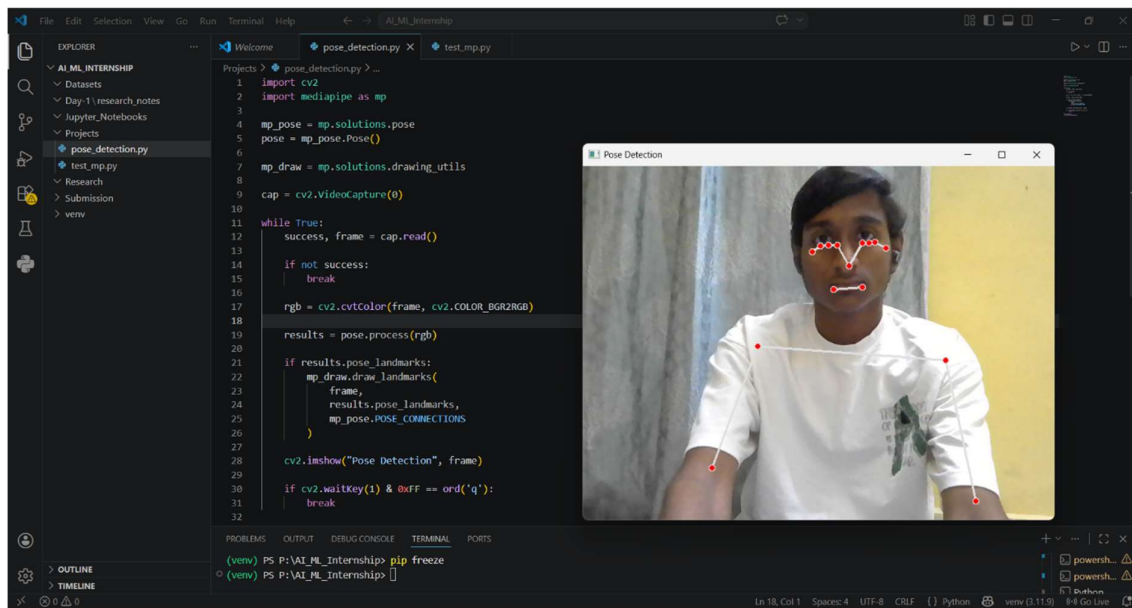
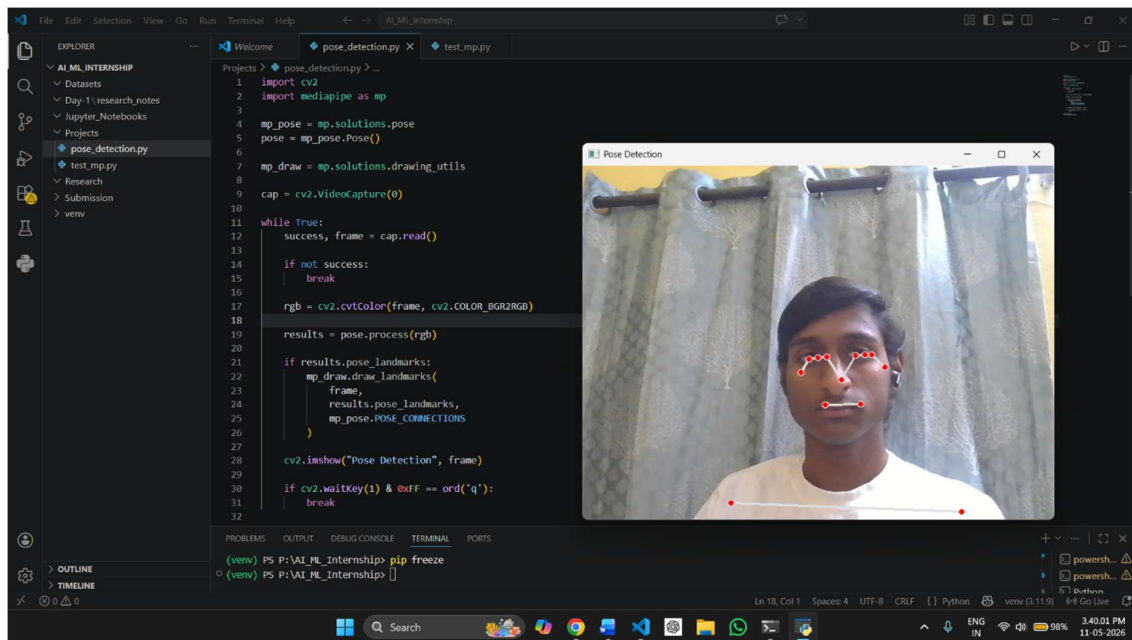
```



8. Output / Results

The pose detection system successfully:

- opened the webcam
- detected body landmarks
- tracked movement in real time
- displayed skeletal connections dynamically



9. Challenges Faced and Solutions

Challenge	Solution
Storage issues in C drive	Shifted project workspace to P drive
MediaPipe installation errors	Reinstalled compatible versions
Virtual environment confusion	Activated venv correctly
OpenCV import issue	Installed inside venv
Pose detection errors	Debugged Python environment

10. Learning Outcomes

During Day 2, the following concepts and skills were learned:

- AI environment setup
- Virtual environment management
- OpenCV basics
- MediaPipe pose tracking
- Human body landmarks
- Real-time webcam processing

11. Conclusion

Day 2 provided practical exposure to computer vision and pose estimation technologies. A real-time pose detection system was successfully implemented using MediaPipe and OpenCV. The task improved understanding of body tracking, pose landmarks, AI environment management, and real-time human movement analysis.